

Homework 2: Higher-Order Functions

- Due: Thursday 09/10 @ 11:59pm
- Points: 1
- Download: hw02.zip

To receive credit, you must solve each problem and then complete a short checkoff interview about your solution. You may use Preceptor for the interview or come to office hours to be interviewed by a member of the course staff. Staff will be doing in-person checkoffs for up to 3 business days after the assignment deadline (including any approved extension). If a checkoff happens more than 3 business days after the regular deadline, staff may ask to confirm your extended deadline via Flexextensions. You will submit a Provenance bundle that includes a record of how you used VS Code, including interactions with Preceptor. Please do not use AI tools other than Preceptor for this assignment.

Readings:

- 1.6: Higher-Order Functions

Getting Started Videos:

If you feel stuck, try watching some walkthrough videos.

Required Questions

Several doctests refer to these functions:

```
def square(x):
    return x * x

def identity(x):
    return x

def triple(x):
    return 3 * x

def increment(x):
    return x + 1
```

Q1: Product

Write a function called `product` that returns the product of the first `n` terms of a sequence. Specifically, `product` takes in an integer `n` and `term`, a single-argument function that determines a sequence. (That is, `term(i)` gives the *i*th term of the sequence.) `product(n, term)` should return `term(1) * ... * term(n)`.

```
def product(n, term):
    """Return the product of the first n terms in a sequence.

    n: a positive integer
    term: a function that takes an index as input and produces a term

    >>> product(3, identity) # 1 * 2 * 3
    6
    >>> product(5, identity) # 1 * 2 * 3 * 4 * 5
    120
    >>> product(3, square)   # 1^2 * 2^2 * 3^2
    36
    >>> product(5, square)   # 1^2 * 2^2 * 3^2 * 4^2 * 5^2
    14400
    >>> product(3, increment) # (1+1) * (2+1) * (3+1)
    24
    >>> product(3, triple)   # 1*3 * 2*3 * 3*3
    162
    """
    """
    *** YOUR CODE HERE ***
    """

python3 -m pytest -k product
```

Q2: Accumulate

There is just one Preceptor interview for this whole question that appears once you have passed the tests for all three of these functions.

Let's take a look at how product is an instance of a more general function called accumulate, which we would like to implement:

```
def accumulate(fuse, start, n, term):
    """Return the result of fusing together the first n terms in a sequence
    and start. The terms to be fused are term(1), term(2), ..., term(n).
    The function fuse is a two-argument commutative & associative function.

    >>> accumulate(add, 0, 5, identity) # 0 + 1 + 2 + 3 + 4 + 5
    15
    >>> accumulate(add, 11, 5, identity) # 11 + 1 + 2 + 3 + 4 + 5
    26
    >>> accumulate(add, 11, 0, identity) # 11 (fuse is never used)
    11
    >>> accumulate(add, 11, 3, square) # 11 + 1^2 + 2^2 + 3^2
    25
    >>> accumulate(mul, 2, 3, square) # 2 * 1^2 * 2^2 * 3^2
    72
    >>> # 2 + (1^2 + 1) + (2^2 + 1) + (3^2 + 1)
    >>> accumulate(lambda x, y: x + y + 1, 2, 3, square)
    19
    """
    """*** YOUR CODE HERE ***"""
```

accumulate has the following parameters:

- fuse: a two-argument function that specifies how the current term is fused with the previously accumulated terms
- start: value at which to start the accumulation
- n: a non-negative integer indicating the number of terms to fuse
- term: a single-argument function; term(i) is the ith term of the sequence

Implement accumulate, which fuses the first n terms of the sequence defined by term with the start value using the fuse function.

For example, the result of accumulate(add, 11, 3, square) is

```
add(11, add(square(1), add(square(2), square(3)))) =
  11 +      square(1) +      square(2) + square(3)   =
  11 +    1          +    4          + 9          = 25
```

Assume that fuse is commutative, fuse(a, b) == fuse(b, a), and associative, fuse(fuse(a, b), c) == fuse(a, fuse(b, c)).

```
python3 -m pytest -k accumulate
```

Then, implement summation (from lecture) and product as one-line calls to accumulate.

Important: Both summation_using_accumulate and product_using_accumulate should be implemented with a single line of code starting with return.

```
def summation_using_accumulate(n, term):
    """Returns the sum: term(1) + ... + term(n), using accumulate.

    >>> summation_using_accumulate(5, square) # square(1) + square(2) + ... + square(4) +
    square(5)
    55
    >>> summation_using_accumulate(5, triple) # triple(1) + triple(2) + ... + triple(4) +
    triple(5)
    45
    >>> # This test checks that the body of the function is just a return statement.
    >>> import inspect, ast
    >>> [type(x).__name__ for x in ast.parse(inspect.getsource(summation_using_accumulate)).
    body[0].body]
    ['Expr', 'Return']
    """
    return ____

def product_using_accumulate(n, term):
    """Returns the product: term(1) * ... * term(n), using accumulate.
```

```

>>> product_using_accumulate(4, square) # square(1) * square(2) * square(3) * square(4)
576
>>> product_using_accumulate(6, triple) # triple(1) * triple(2) * ... * triple(5) *
triple(6)
524880
>>> # This test checks that the body of the function is just a return statement.
>>> import inspect, ast
>>> [type(x).__name__ for x in ast.parse(inspect.getsource(product_using_accumulate)).
body[0].body]
['Expr', 'Return']
"""
return ____

python3 -m pytest -k summation_using_accumulate

python3 -m pytest -k product_using_accumulate

```

Q3: Make Repeater

Implement the function `make_repeater` which takes a one-argument function `f` and a positive integer `n`. It returns a one-argument function so that `make_repeater(f, n)(x)` returns the value of `f(f(...f(x)...))`, in which `f` is applied `n` times to `x`. For example, `make_repeater(square, 3)(5)` squares 5 three times and returns 390625, just like `square(square(square(5)))`.

```

def make_repeater(f, n):
    """Returns the function that computes the nth application of f.

    >>> add_three = make_repeater(increment, 3)
    >>> add_three(5)
    8
    >>> make_repeater(triple, 5)(1) # 3 * (3 * (3 * (3 * (3 * 1))))
    243
    >>> make_repeater(square, 2)(5) # square(square(5))
    625
    >>> make_repeater(square, 3)(5) # square(square(square(5)))
    390625
    """
    """ YOUR CODE HERE """

python3 -m pytest -k make_repeater

```

Q4: Composite Identity Function

Write a function that takes in two single-argument functions, `f` and `g`, and returns another function that has a single parameter `x`. The returned function should return `True` if `f(g(x))` is equal to `g(f(x))` and `False` otherwise. You can assume the output of `g(x)` is a valid input for `f` and vice versa.

```

def composite_identity(f, g):
    """Return a function with one parameter x that returns True if f(g(x)) is
    equal to g(f(x)). You can assume the result of g(x) is a valid input for f
    and vice versa.

    >>> add_one = lambda x: x + 1 # adds one to x
    >>> square = lambda x: x**2 # squares x [returns x^2]
    >>> b1 = composite_identity(square, add_one)
    >>> b1(0) # (0 + 1) ** 2 == 0 ** 2 + 1
    True
    >>> b1(4) # (4 + 1) ** 2 != 4 ** 2 + 1
    False
    """
    """ YOUR CODE HERE """

python3 -m pytest -k composite_identity

```

Submit

Run Provenance: Prepare Submission Bundle from the VS Code command palette to create your submission zip, and upload that zip to Gradescope. For a refresher on how to do this, refer to Lab 00.

Just For Fun Questions

The questions below are out of scope for 61A. You can try them if you want an extra challenge, but they're just puzzles that are not required for the course. Almost all students will skip them, and that's fine. We will not be prioritizing support for these questions on Ed or during Office Hours.

Q5: Church Numerals

The logician Alonzo Church invented a system of representing non-negative integers entirely using functions. The purpose was to show that functions are sufficient to describe all of number theory: if we have functions, we do not need to assume that numbers exist, but instead we can invent them.

Your goal in this problem is to rediscover this representation known as Church numerals. Church numerals are a way to represent non-negative integers via repeated function application. Specifically, Church numerals (such as zero, one, and two below) are functions that take in a function f and return a new function which, when called, repeats f a number of times on some argument x . Here are the definitions of zero, as well as a successor function, which takes in a Church numeral n as an argument and returns a function that represents the Church numeral one higher than n :

```
def zero(f):
    return lambda x: x

def successor(n):
    return lambda f: lambda x: f(n(f)(x))
```

First, define functions `one` and `two` such that they have the same behavior as `successor(zero)` and `successor(successor(zero))` respectively, but do not call `successor` in your implementation.

Next, implement a function `church_to_int` that converts a Church numeral argument to a regular Python integer.

Finally, implement functions `add_church`, `mul_church`, and `pow_church` that perform addition, multiplication, and exponentiation on Church numerals.

```
def one(f):
    """Church numeral 1: same as successor(zero)"""
    """ YOUR CODE HERE """

def two(f):
    """Church numeral 2: same as successor(successor(zero))"""
    """ YOUR CODE HERE """

three = successor(two)

def church_to_int(n):
    """Convert the Church numeral n to a Python integer.

    >>> church_to_int(zero)
    0
    >>> church_to_int(one)
    1
    >>> church_to_int(two)
    2
    >>> church_to_int(three)
    3
    """
    """ YOUR CODE HERE """

def add_church(m, n):
    """Return the Church numeral for m + n, for Church numerals m and n.

    >>> church_to_int(add_church(two, three))
    5
    """
    """ YOUR CODE HERE """

def mul_church(m, n):
    """Return the Church numeral for m * n, for Church numerals m and n.

    >>> four = successor(three)
    >>> church_to_int(mul_church(two, three))
    6
    >>> church_to_int(mul_church(three, four))
    12
    """
    """ YOUR CODE HERE """

def pow_church(m, n):
```

```
"""Return the Church numeral m ** n, for Church numerals m and n.

>>> church_to_int(pow_church(two, three))
8
>>> church_to_int(pow_church(three, two))
9
"""
*** YOUR CODE HERE ***
```

This command tests them all:

```
python3 -m pytest -k church
```

You can also test each part individually:

```
python3 -m pytest -k church_to_int
```

```
python3 -m pytest -k add_church
```

```
python3 -m pytest -k mul_church
```

```
python3 -m pytest -k pow_church
```